Knowledge-Based Systems 317 (2025) 113396

Contents lists available at ScienceDirect

Knowledge-Based Systems

journal homepage: www.elsevier.com/locate/knosys

# Harnessing code domain insights: Enhancing programming Knowledge Tracing with Large Language Models

Xinjie Sun [a,b,d], Qi Liu [c,d], Kai Zhang [b,d], Shuanghong Shen [e,*], Lina Yang [a], Hui Li [a]

[a] School of Computer Science, Liupanshui Normal University, Liupanshui, China
[b] School of Computer Science and Technology, University of Science and Technology of China, Hefei, China
[c] School of Artificial Intelligence and Data Science, University of Science and Technology of China, Hefei, China
[d] State Key Laboratory of Cognitive Intelligence, Hefei, China
[e] Institute of Artificial Intelligence, Hefei Comprehensive National Science Center, Hefei, China

ARTICLE INFO

*Keywords:*
Knowledge tracing
Data mining
Intelligent tutoring systems
Large language models
Efficiency and interpretability

ABSTRACT

Knowledge Tracing (KT) evaluates students' mastery of knowledge by analyzing their historical interactions with exercises and predicts their performance on subsequent tasks. Although traditional KT methods have begun to focus on the assessment of programming skills, they are limited by the bottleneck of manually annotating knowledge concepts (KCs) and the inadequacy of constructing relationships between these points. To address this issue, we propose a ***K**nowledge **T**racing method **E**nhanced by the powerful **C**ode insight capabilities of Large Language Models* ***(CEKT)***. Specifically, we designed three different prompt tuning strategies for Large Language Models (LLMs) to comprehensively construct Q-matrices that cover KCs and their relationships across various programming domains and exercises. Additionally, we developed a knowledge graph integrating three dimensions to express the complex relationships between KCs in a fine-grained manner, thereby providing a more accurate assessment of students' knowledge mastery. Furthermore, we established a graph attention network among KCs to promote interaction between representations of similar syntactic KCs, enhancing the inference capability of students' programming knowledge state and the effectiveness of KT. Through this approach, we achieved high-quality and interpretable knowledge state inference and demonstrated outstanding performance in predicting student outcomes. Our work highlights a potential future research direction for prompt-tuned LLMs in the KT domain, emphasizing high interpretability and efficiency. For broader research purposes, we have prepared to release our data and source code at https://github.com/xinjiesun-ustc/CEKT, encouraging further innovation in this field.

## 1. Introduction

Knowledge Tracing (KT) assesses students' mastery of knowledge concepts (KCs) based on their historical answering records and uses these assessments to predict their future performance [1–3], as illustrated in Fig. 1. As programming skills become increasingly important, more people are encouraged to enhance their abilities through programming exercises [4,5]. In this context, monitoring learners' programming knowledge and abilities becomes crucial [6].

In recent years, deep learning has made significant advancements in the field of KT for academic subjects, utilizing various advanced methods, including autoregressive deep sequential models [7,8], attention mechanisms [2,9,10], and graph-based KT models [11,12]. Recently, many studies have also begun to focus on programming KT tasks. For instance, Code-DKT [13] uses Code2Vec technology to transform students' code into embedding vectors and integrates them with the DKT model to model their knowledge state. PST [14] evaluates students' programming skills by analyzing KCs during the programming process. OKT [15] explores open KT to predict students' code generation abilities. PTDK [16] uses dual-sequence modeling techniques to predict students' Subsequently code submission performance. These studies demonstrate the powerful potential of deep learning models in programming education, providing strong support for future educational assessments and personalized learning.

Although deep learning has garnered some attention in the field of programming KT in recent years, current research still tends to directly treat exercises as KCs due to the bottleneck of manual annotation [14,17,18]. This approach limits the model's ability to capture finer-grained

* Corresponding author.
*E-mail addresses:* xinjiesun@mail.ustc.edu.cn (X. Sun), qiliuql@ustc.edu.cn (Q. Liu), kkzhang08@ustc.edu.cn (K. Zhang), closer@mail.ustc.edu.cn (S. Shen), linayang@lpssy.edu.cn (L. Yang), lihui@lpssy.edu.cn (H. Li).

https://doi.org/10.1016/j.knosys.2025.113396
Received 29 July 2024; Received in revised form 9 March 2025; Accepted 19 March 2025
Available online 4 April 2025
0950-7051/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

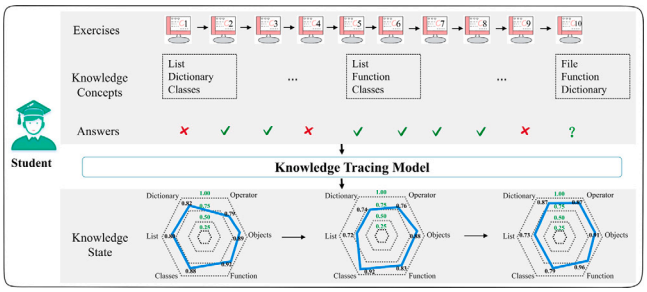



**Fig. 1.** A toy example demonstrating the process of programming KT.

KCs. Unlike general academic exercises (e.g., multiple-choice exercises in mathematics), the proficiency of programming learners is reflected not only in whether they correctly answer exercises but also in their specific mastery of the many KCs involved in each exercise, as illustrated by the radar chart in Fig. 1 showing the degree of mastery of KCs. Therefore, efficiently and accurately extracting KCs from programming exercises and reasonably utilizing these KCs to assess students' programming abilities has become a key challenge in programming KT research. This paper aims to address this issue by proposing innovative methods to enhance the efficiency and accuracy of KC extraction, thereby providing a more comprehensive assessment of students' programming abilities.

To accurately measure students' proficiency in KCs in programming education, we must address three major challenges. **First**, it is essential to comprehensively construct a KC system that covers different programming domains and effectively extract the specific KCs involved in each exercise. This requires an in-depth analysis of exercises to accurately identify their KCs. **Second**, the relationships between KCs are intricate and complex. Exploring both strong and weak associations between KCs from multiple dimensions is a significant challenge for assessing students' knowledge state. Understanding these associations not only helps evaluate students' current knowledge state but also predicts their future learning performance. **Finally**, effectively utilizing the knowledge associations between different programming exercises and constructing high-precision syntactic knowledge representations based on these associations are key to improving the accuracy and practicality of models. This requires meticulous syntactic and semantic analysis of programming exercises and the construction of a high-precision model that reflects the associations between the KCs of the exercises.

To effectively address the aforementioned challenges, we propose the Code-Enhanced KT (CEKT) model, which incorporates several key innovations to improve the effectiveness, efficiency, and interpretability of KT in programming exercises. **First**, we meticulously designed three different prompts and applied them to LLMs to comprehensively construct KCs covering various programming domains and exercises. These prompts respectively generate domain KCs, exercise-specific KCs, and the Q-matrix representing the relationships between exercises and KCs. **Second**, we constructed a complex knowledge graph using parent–child, sibling, and parent–parent relationships to finely express the intricate relationships between KCs, thereby enabling a more precise assessment of students' knowledge mastery. **Finally**, we established a graph attention network among KCs to facilitate interactions between similar syntactic knowledge representations. This network enhances the inference capability of students' programming knowledge state and the effectiveness of KT. Through these innovations, the CEKT model achieves comprehensive and accurate assessments of students' programming knowledge state, providing a solid foundation for personalized teaching and learning path planning.

In this study, we propose a novel model framework to address the exercise of KT in programming exercises, aiming to evaluate students' programming knowledge mastery more quickly and comprehensively. Our main contributions are summarized as follows:

- We propose an innovative assessment method that leverages the powerful code processing capabilities of LLMs to enhance the effectiveness of KT in the programming domain.
- We construct a KT model that combines LLMs prompting techniques and graph attention networks (GAT). The model is not in the form of a fine-tuned version but is developed based on data generated by LLMs. By constructing a complex programming knowledge graph, this model significantly improves the inference and tracking of students' programming knowledge state and their interpretability, particularly enhancing educational quality and equity in resource-constrained settings.
- In comparative experiments with existing KT models, our CEKT model demonstrated superior performance across multiple programming domain datasets, validating its effectiveness and educational value in practical applications.

These contributions not only showcase the innovation of our research but also provide powerful tools for programming education practice, helping educators better understand and support students' learning processes.

## 2. Related works

Accurately identifying students' programming skills during programming exercises is a highly valuable task. Subsequently, we will introduce related research from the perspectives of KT and LLMs.

### 2.1. Knowledge tracing

In the task of KT, the core objective is to accurately track and predict students' mastery of knowledge [19]. In recent years, the application of deep learning technologies in this field has made significant progress [20]. By introducing complex model structures such as Long Short-Term Memory (LSTM) networks [1], Memory Networks [21–23], and Attention Mechanisms [2,9,10], researchers have been able to more precisely distinguish students' understanding of different KCs. These advancements not only improve the accuracy of the models but also enhance insights into students' learning processes. Additionally, by incorporating features like exercise-solving characteristics and exercise IDs, the models can more comprehensively consider students' exercise-solving processes, thus more accurately predicting their knowledge state [24,25].

Integrating educational theories into deep learning models has also become an important research direction. By simulating student behavior patterns, learning gains, and forgetting processes, these models not only enhance the understanding of students' learning processes but also improve the interpretability of the models [26]. By analyzing students' responses to different hierarchies of exercises, researchers can accurately identify students' target hierarchies and conduct more fine-grained knowledge state modeling [3].

In the field of programming education, early research primarily relied on course exam scores or learner log data to identify potential learning obstacles, but these methods often overlooked the code submitted by students. Recent studies have started to focus on assessing students' programming skills by analyzing their code. For instance, the Code-DKT model embeds students' code into Code2Vec and then inputs it into the DKT model for modeling, achieving good results [13]. The PST model extracts implicit knowledge in students' programming processes to achieve more fine-grained skill assessments [14]. The OKT model is the first to explore open-ended KT, combining program synthesis methods and student KT methods to predict students' code generation [15]. The PTDK model considers multiple possible detections of students' code and uses a dual-sequence modeling approach to predict the outcome of the Subsequently submission [16].

In light of the aforementioned, the KT task has made significant progress. By combining deep learning technologies and educational

theories, researchers can more accurately track and predict students' knowledge state, providing strong data support for personalized learning. However, KT in the field of programming has received relatively less attention, especially since most datasets in the programming field lack corresponding KCs. As a result, most models use exercises to represent KCs, which limits the development of programming KT.

**Graph-based Knowledge Tracing:** Inspired by graphical learning techniques, researchers have begun to apply graph neural network (GNN) technologies to flexibly model the graphical relationships between problems and skills, providing new solutions for knowledge tracing (KT) tasks [11,12,27]. A graph serves as a data structure that models a set of entities (nodes) and their interrelations (edges). In the realm of knowledge components (KCs), a natural graph structure exists, indicating that integrating this structure as supplementary information could enhance the effectiveness of KT tasks. The graph-based framework conceptualizes the potential graph structure of the KCs as a graph $G = (V, E)$, where the nodes $V = \{v_1, v_2, \dots, v_N\}$ correspond to the set of KCs, and the edges $E \subseteq V \times V$ represent the relationships among these KCs. Additionally, $h_t = \{h_{ti} \in V\}$ denotes the student's knowledge state at time $t$ after responding to an exercise. **Our CEKT model also employs graph network technology to describe the relationships between KCs in students' programming exercises.**

### 2.2. Applications of LLMs in the field of programming

In recent years, the application of LLMs in the field of programming has made significant progress. Below is a detailed introduction to the work related to LLMs in code generation, code understanding, and programming education.

**Code Generation:** In the realm of natural language programming, the CoRE system utilizes LLMs to interpret and execute natural language instructions, unifying the representations of natural language programming, pseudocode programming, and flow programming. It also compensates for the shortcomings of LLMs in specific domains by calling external tools [28]. Additionally, the Low-code LLMs framework provides a more controllable and stable response through low-code visual programming interaction, enhancing the friendliness and applicability of human–computer interaction [29]. In terms of generating programming examples, LLMs are used to create interactive working examples that help students understand the process of solving programming exercises [30]. In program generation tasks, LLMs often struggle to produce entirely correct programs when dealing with unfamiliar domain-specific languages (DSLs). To address this, a hybrid approach has been proposed, where a task-specific context-free model generated by an LLMs guides program synthesis, thereby improving the accuracy of program generation [31].

**Code Understanding:** LLMs also show significant advantages in code understanding. For example, one study demonstrated how an LLMs-based conversational user interface embedded in an integrated development environment (IDE) can help developers explain code, provide API call details, explain key domain-specific terms, and offer examples of API usage [32]. Additionally, the Copilot evaluation tool assesses the performance of LLMs in IDEs through static and execution success metrics, covering tasks such as code generation, documentation generation, test case generation, error fixing, and workspace understanding [33]. In robot programming, research found that LLMs tend to forget critical information from user prompts during execution, proposing methods to reduce execution errors through prompt engineering [34].

**Programming Education:** In the field of programming education, the CodeAid system provides immediate support in programming courses by answering conceptual exercises, generating pseudocode, and annotating students' erroneous code, thereby avoiding directly providing code solutions and promoting deep learning among students [35]. LLMs are also used to generate code tracing exercises, demonstrating their potential in generating diverse code tracing exercises [36]. Furthermore, the ECKT framework leverages LLMs to generate detailed exercise descriptions and KCs, enhancing the assessment of students' programming abilities [37]. The SINKT model improves the accuracy of KT by introducing LLMs to construct structural relationship graphs of concepts and exercises [38]. In actual teaching, LLMs are used to generate programming assignments and exam exercises, and are fully integrated into courses to reduce the emphasis on syntax and code writing, focusing more on explaining code, testing code, and decomposing complex exercises [39]. Research also found that students exhibit over-reliance and self-regulation issues when using LLMs for programming learning [40]. In generating exercises related to students' code, LLMs have shown potential in code understanding and generation, but also revealed issues similar to novice programming errors [41]. **Our CEKT model uses LLMs to generate domain-specific KCs and their relationships for programming KT.**

Despite the strong capabilities demonstrated by LLMs in the field of programming, their limitations cannot be overlooked. When handling complex tasks, LLMs often require meticulous prompt engineering and support from external tools to enhance the stability and accuracy of their responses [29,42]. In summary, the application of LLMs in the programming domain brings new opportunities and challenges for code generation, code understanding, and programming education.

**Table 1**
Mathematical symbol and descriptions.

| Symbol | Description |
|---|---|
| S, E | The set of student and programming exercise |
| KC, R | The set of KCs and response |
| $L$ | Students' exercise sequence |
| $h_t$ | Students' knowledge state at time t |
| $Q$ | Exercise and KCs Relationship Matrix |
| $r$, $\mathbf{R}$ | Submitted code acceptance and embedding |
| n, m, k | The number of student, exercise and kc |
| $d_r, d_k$ | Dimensions of the learner's response and KC, and exercise |
| $d_h, d_e$ | Dimensions of hidden knowledge state and exercise |

## 3. Preliminary

In this section, we will provide the formal definition of the KT method enhanced by LLMs. Additionally, we will introduce some important student behavior embeddings involved in our proposed CEKT framework. Table 1 summarizes the mathematical symbols used in this paper.

### 3.1. Problem definition

In an online judging system, the set of students is represented by $S$, the set of coding exercises is denoted by $E$,[1] and the set of student responses is denoted by $R$. The set of students $S = \{s_1, s_2, \dots, s_n\}$ contains $n$ different students. The set of coding exercises $E = \{e_1, e_2, \dots, e_m\}$ contains $m$ different coding exercises. The set of KCs involved in the exercises, $KC = \{kc_1, kc_2, \dots, kc_k\}$, includes $k$ different KCs. Each student can have $L$ ($L \geq 1$) records of responses to coding exercises. Specifically, for multiple submissions of the same exercise, we only select the result of the first submission as the record. The set of response labels $R = \{0, 1\}$ is a set containing two elements, used to indicate whether the student's submitted code was accepted by the system. 1 represents that the code was fully accepted by the system, while 0 represents any other situation besides acceptance.

Therefore, throughout the process of student programming tests, to better understand the students' specific programming behavior, we represent it as a detailed sequence of coding events $L = \{(e_1, kc_1, r_1), (e_2, kc_2, r_2), \dots, (e_t, kc_t, r_t)\}$. Under these circumstances, $e_t$ represents the

[1] In this paper, the corresponding KCs are generated by LLMs based on the coding exercises

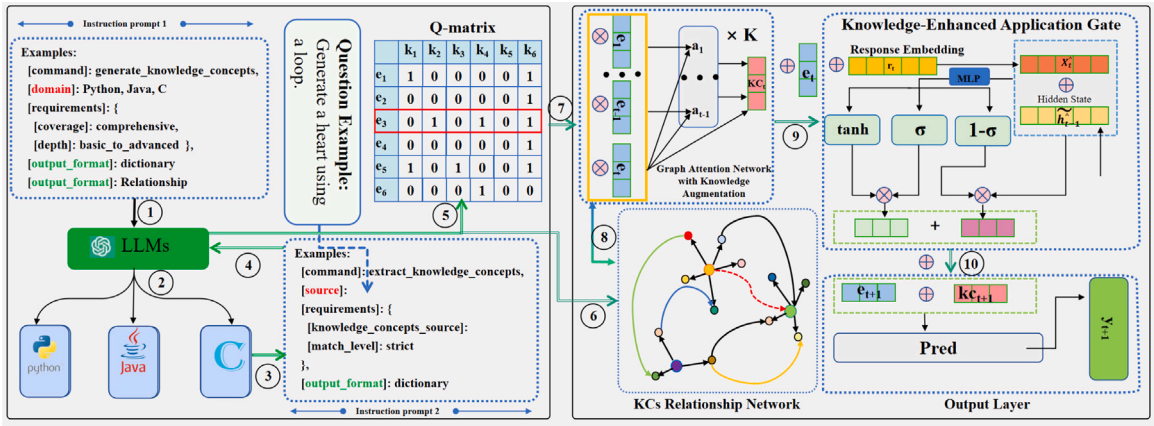



**Fig. 2.** The core architecture of CEKT model: Enhancing KT with LLMs to Predict Student Performance at Time Step $t+1$. Inputs include code $e_t$, KCs $kc_t$, student responses $r_t$, and knowledge state $h_t$ at time step $t$.

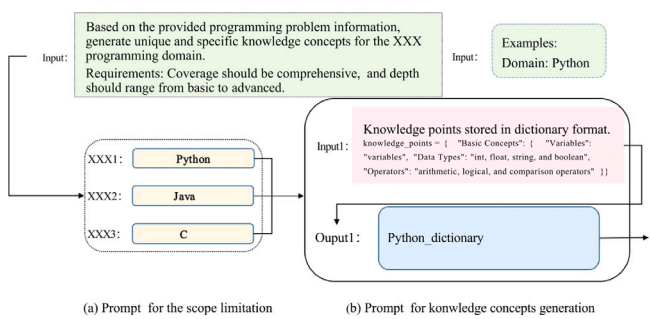



**Fig. 3.** Examples of constructing code domain KCs using prompts, with the same prompt construction methods applied to all three datasets.

coding exercise answered by the student at time step $t$, $kc_t$ represents the corresponding KC generated for the exercise using the capabilities of the large model, and $r_t$ represents the response label of the student's answer to coding exercise $e_t$ at time step $t$, i.e., the correctness of the answer. Additionally, a Q-matrix is used to represent the relationship between exercises and KCs. The Q-matrix of exercises is known (generated by the LLMs in this paper) and is denoted as $Q = Q_{ij}^{m\times k}$. When exercise $e_i$ involves KC $kc_j$, $Q_{ij} = 1$; otherwise, $Q_{ij} = 0$. The length of such a practice sequence is $t$. By analyzing students' historical answer sequences, we can track their mastery of programming knowledge. The definition of KT in programming practice in this paper is as follows:

**Knowledge Tracing (KT)**: Given a student's exercise records $L = \{(e_1, kc_1, r_1), (e_2, kc_2, r_2), \ldots, (e_t, kc_t, r_t)\}$, the task of KT enhanced by LLMs is a dynamic process. A student's level of programming knowledge continuously evolves with ongoing practice. The goal of this task is to assess their mastery of programming knowledge based on their historical code submissions and predict their performance on the Subsequently coding exercise. At various stages and for students with different levels of programming skills, we provide appropriate feedback and guidance. For example, for students who excel in programming, we can acknowledge their achievements and offer them more challenging coding tasks. This way, we can effectively promote student development and help them continuously improve their programming skills.

### 3.2. Embeddings

Before delving into the interactions of KT, we need to briefly introduce some fundamental elements, primarily involving the basic embeddings of CEKT. This helps to better understand the internal working mechanisms of CEKT.

In the CEKT model, we focus on three key elements: exercise, KCs, and student response, represented through embedding matrices. Specifically, we use the embedding matrix $\mathbf{E} \in \mathbb{R}^{m\times d_e}$ to represent all exercises, with dimensions $m \times d_e$, where $m$ is the number of exercises and $d_e$ is the size of the embedding dimension. The embedding matrix $\mathbf{KC} \in \mathbb{R}^{k\times d_k}$ represents all KCs associated with the exercises, with dimensions $k \times d_k$, where $k$ is the number of KCs and $d_k$ is the size of the embedding dimension. The embedding matrix $\mathbf{R} \in \mathbb{R}^{2\times d_r}$ represents student responses, with dimensions $2\times d_r$, where 2 indicates the correctness of the response (1 for correct, 0 for incorrect), and $d_r$ is the size of the embedding dimension. These embedding matrices allow us to better analyze and diagnose students' cognitive states.

## 4. The CEKT framework

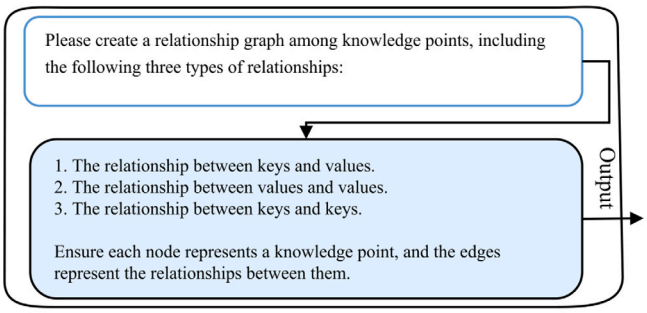
In this section, we will delve into the proposed CEKT framework, whose structure is illustrated in Fig. 2. The framework is divided into two main parts. The first part involves the generation and extraction of KCs from programming exercises using a LLMs, which then constructs a Q-matrix and a KC relationship graph. The second part, based on the LLMs, performs knowledge-enhanced KT to more accurately assess students' knowledge mastery. Subsequently, we will discuss and explain these two modules in more detail.

### 4.1. Constructing KCs in the code domain

Different programming languages contain their own unique programming KCs, and traditional manual annotation methods can often be cumbersome and inefficient when handling these KCs [43]. By leveraging the powerful programming capabilities of LLMs (such as GPT-4.0), we can generate high-quality KCs and build a knowledge network among these concepts. To ensure the accuracy of the generated KCs, we impose strict constraints on the prompts to prevent the LLMs from generating incorrect concepts within its knowledge blind spots. Specifically, we generate corresponding KCs for the code domains of Python, Java, and C languages through instructional prompting.

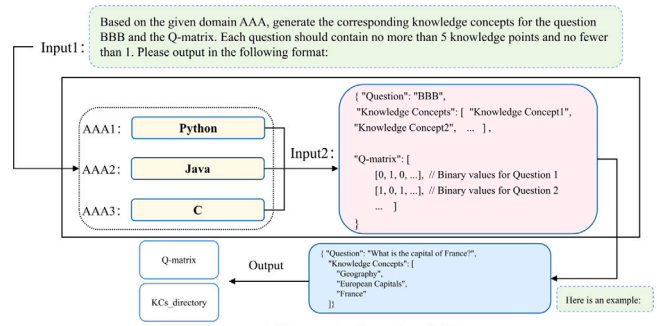
Inspired by the methodologies of [44,45], we adopted the prompting technique, a popular LLMs prompting method, and made adaptive adjustments to it. As shown in Fig. 3, (a) is "scope-limited prompting", which restricts the entire knowledge construction scope through hard prompts; (b) is "format-constrained output prompting", which restricts the format output by providing the LLMs with a specific generation example.

$$K = \{D_i \mid D_i = \{P_{k,v} \mid P_{k,v} = (\text{key}, \text{value})\}\},\quad i \in \{Python, Java, C\}, \tag{1}$$




(a) Prompt for the Relationship

**Fig. 4.** Examples of generating exercise KCs using prompts, with the same prompt construction methods applied to all three datasets.




(a) Prompt for Q-matrix and KCs

**Fig. 5.** Examples of constructing Q-matrices for exercises and corresponding KC using prompts, with the same prompt construction methods applied to all three datasets.

Where $K$ represents the entire KC dictionary, each $D_i$ represents a specific knowledge domain, and each $P_{k,v}$ denotes the relationship between parent and child KCs, where the key is the parent KC and the value is the child KC.

To construct a relationship graph among KCs, we need to define sets of nodes and edges, and specify three particular types of relationships: relationships between keys and values, relationships between values, and relationships between keys. We utilized an instruction prompt and made adaptive adjustments, as shown in Fig. 4. First, we define the set of nodes $V$, which includes all KCs, encompassing both keys and values. Subsequently, we define the set of edges $E$, which represents the three types of relationships:

- **Relationships between Keys and Values:** This represents the relationship between each parent KC and its child KCs.
- **Relationships between Values:** This represents the interrelationships among child KCs.
- **Relationships between Keys:** This represents the interrelationships among parent KCs.

The construction of edges corresponding to domain knowledge $D_i$ extracted from the KC dictionary $K$ is as follows:

$$\begin{aligned} E_{kv} &= \{(k, v) \mid k, v \in P_{k,v}\}, \\ E_{vv} &= \{(v_1, v_2) \mid v_1, v_2 \in V\}, \\ E_{kk} &= \{(k_1, k_2) \mid k_1, k_2 \in K\}, \end{aligned} \tag{2}$$

we merge these sets of edges to obtain the complete set of edges $E$. Therefore, the entire relationship graph can be represented as a directed graph $G = (\mathcal{N}, E)$, where $\mathcal{K}$ is the set of keys, $\mathcal{V}$ is the set of values, and $E$ is the set of edges representing the three specific types of relationships. This graph structure clearly shows the relationships among KCs, making it easier to analyze and understand.

$$\begin{aligned} E &= E_{kv} \cup E_{vv} \cup E_{kk}, \\ \mathcal{N} &= \mathcal{K} \cup \mathcal{V}, \\ G &= (\mathcal{N}, E), \end{aligned} \tag{3}$$

where $\mathcal{N}$ represents the set of all nodes.

### 4.2. Constructing the KCs for exercises

In the cognitive-level research paradigm, the Q-matrix is a key tool used to automatically represent the association between exercises and KCs. Each exercise's relevant KCs are automatically captured through a Q-matrix generated by LLMs. To construct an accurate and effective Q-matrix $Q$, we utilize an automated process driven by LLMs, as illustrated in Fig. 5. Specifically, we used three common student answer datasets from the AIZU online platform: AIZU_Python, AIZU_Java, and AIZU_C. For each set of exercises $E$, we input each exercise $e$ into a LLMs to generate the corresponding set of KCs $kc$ for that exercise. We strictly ensure that these KCs must come from the corresponding set of KCs $D_i$, ensuring $kc \subseteq D_i$. By batch processing these data, we generate domain-specific KCs for each exercise and construct the Q-matrix representing the relationship between KCs and exercises. The calculation process is as follows:

$$Q_{tj} = \begin{cases} 1 & \text{if } d_j \in kc, \\ 0 & \text{if } d_j \notin kc, \end{cases} \tag{4}$$

where the $Q$ matrix is a binary matrix of size $m \times k$ (where $m$ is the number of exercises and $k$ is the number of KCs), $d_j$ represents the $j$th KC, $e_t$ represents the $t$th exercise, and $kc_t$ represents the set of KCs contained in exercise $e_t$. The KC relevance vector for each exercise $e$ can be directly represented using the corresponding row vector $\mathbf{Q}_e$ in the $Q$ matrix, i.e.,

$$\mathbf{Q}_e = \mathbf{x}_e \times Q, \tag{5}$$

where $\mathbf{x}_e \in \{0, 1\}^{1\times m}$ is the one-hot encoding vector for the ID of exercise $e$, and $\mathbf{Q}_e \in \{0, 1\}^{1\times k}$ represents the KC relevance vector for that exercise.

### 4.3. Knowledge-enhanced GAT

The Q-matrix indicates the relationships between exercises and KCs, while the graph $G$, obtained through a LLMs, depicts the interrelationships among the KCs. Leveraging these, we can employ a Graph Attention Network (GAT) to construct a feature-aggregated, enhanced knowledge network structure. This approach helps the model better represent the strength of relationships and characteristics of KCs, thereby improving the assessment of knowledge mastery. Specifically, we build a subgraph for time-series KC feature aggregation based on the directed graph $G = (\mathcal{N}, E)$, where $\mathcal{N}$ is the set of all nodes and $E$ is the set of all edges. At each time step $t$, a KC can only be associated with KCs from time $t-1$ and earlier. We utilize GAT for feature aggregation. The detailed steps are as follows.

Given the temporal modeling approach in KT, a subgraph needs to be constructed during knowledge enhancement, which includes relationships between KCs in the current and previous exercises. Thus, at time $t$, we construct a subgraph $G_t = (\mathcal{N}_t, E_t)$, where $\mathcal{N}_t$ is the set of nodes at time $t$ and earlier, specifically $\mathcal{N}_t = \{v_0, v_1, \ldots, v_t\}$. $E_t$ is the set of all valid edges at time $t$ and earlier, i.e., $E_t = \{(v_i, v_j) \in E \mid 0 \le i < j \le t\}$.

The input feature vector of each node $v_i \in \mathcal{N}_t$ corresponding to the KC is denoted as $\mathbf{h^{kc}}_i$, calculated as follows:

$$\mathbf{h^{kc}}_i = \mathbf{Q}_e \times e_t \quad \text{where} \quad \mathbf{h^{kc}}_i \in \mathbb{R}^{d_k}. \tag{6}$$

For each node $v_t$ at time $t$ and its predecessor node $v_s$ ($s < t$), we calculate the attention weight coefficient $e_{ts}$ as follows:

$$e_{ts} = \begin{cases} w_{kv} \cdot \sigma(\mathbf{a}^T[\mathbf{W_1}\mathbf{h^{kc}}_t \parallel \mathbf{W_2}\mathbf{h^{kc}}_s]) & \text{if } (v_t, v_s) \in E_{kv} \cap E_t, \\ w_{vv} \cdot \sigma(\mathbf{a}^T[\mathbf{W_3}\mathbf{h^{kc}}_t \parallel \mathbf{W_4}\mathbf{h^{kc}}_s]) & \text{if } (v_t, v_s) \in E_{vv} \cap E_t, \\ w_{kk} \cdot \sigma(\mathbf{a}^T[\mathbf{W_5}\mathbf{h^{kc}}_t \parallel \mathbf{W_6}\mathbf{h^{kc}}_s]) & \text{if } (v_t, v_s) \in E_{kk} \cap E_t, \end{cases} \tag{7}$$

where $\sigma(\cdot)$ is the LeakyReLU activation function, $w_{kv}, w_{vv}, w_{kk} \in \mathbb{R}$ are trainable scalar weight parameters representing the weights of key–value, value–value, and key–key relationships, respectively. These weights are used for different types of edge relationships. The trainable matrices $\mathbf{W_1}, \mathbf{W_2}, \mathbf{W_3}, \mathbf{W_4}, \mathbf{W_5}, \mathbf{W_6} \in \mathbb{R}^{d \times d_k}$ are responsible for feature transformation. The trainable vector $\mathbf{a} \in \mathbb{R}^{d+d}$ is used to calculate the attention weights, and $\parallel$ denotes the concatenation operation of vectors.

Through this method, we can effectively leverage the relationships between KCs in the temporal sequence, enhancing the model's ability to represent the relationships between KCs and evaluate knowledge mastery. After calculating the attention weight coefficient $e_{ts}$, we need to normalize these weights. We use the softmax function to normalize the weights:

$$\alpha_{ts} = \frac{\exp(e_{ts})}{\sum_{k \in \mathcal{N}_t} \exp(e_{tk})}, \tag{8}$$

where $\alpha_{ts}$ is the normalized attention weight coefficient, representing the attention weight of node $v_t$ toward its immediate predecessor node $v_s$.

Subsequently, we employ the normalized attention weights $\alpha_{ts}$ to derive a weighted average of the features from the predecessor nodes, thereby yielding the refined feature representation $\mathbf{h^{kc}}'_t$ for node $v_t$:

$$\mathbf{h^{kc}}'_t = \sigma\left(\sum_{s \in \mathcal{N}_t} \alpha_{ts} \mathbf{W} \mathbf{h^{kc}}_s\right), \tag{9}$$

where $\mathbf{W} \in \mathbb{R}^{d \times d_k}$ is a trainable weight matrix, and $\mathbf{h^{kc}}'_t$ represents the new feature of node $v_t$.

To enhance the stability and expressiveness of the model, we perform feature aggregation on the neighboring nodes, including the node itself. Specifically, we employ $K$ independent attention heads to compute attention weights and feature representations, followed by concatenating the outputs of these heads. The computation is as follows:

$$\mathbf{h^{kc}}'_t = \parallel_{k=1}^{K} \sigma\left(\sum_{s \in \mathcal{N}_t} \alpha_{ts}^k \mathbf{W}^k \mathbf{h^{kc}}_s\right), \tag{10}$$

where $\mathbf{h^{kc}}'_t$ represents the result after concatenation and activation function processing at time step $t$. The $k$ denotes the $k$th head, and $c$ denotes a specific channel. $\parallel$ is a custom concatenation operator used to concatenate multiple vectors together. $\alpha_{ts}^k$ is the attention weight, representing the attention weight of node $s$ at time step $t$ in the $k$th head. $\alpha_{ts}^k$ reflects the correlation between node $t$ and node $s$.

### 4.4. Knowledge-enhanced application gate

A learner's cognitive state evolves continuously with ongoing practice [46]. To further deepen our understanding of cognitive changes during practice, particularly in measuring knowledge mastery between exercises, we have designed a Knowledge-Enhanced Application Gate (KAG) mechanism to update students' knowledge mastery. Specifically, we first concatenate the aggregated feature of the KCs at time $t$, $\mathbf{h^{kc}}'_t$, with the exercise information $e_t$ and the student's response to the exercise $r_t$, resulting in the student's response information at time $t$, $\hat{X}_t$. The detailed calculation process is as follows:

$$\hat{X}_t = \mathbf{h^{kc}}'_t \oplus e_t \oplus r_t, \tag{11}$$

where $\oplus$ denotes the vector concatenation operation. Subsequently, the current input vector $\hat{X}_t$ is concatenated with the previous hidden state vector $\widetilde{h}_{t-1}$ to obtain the combined input vector $D_t$. To introduce new information while preserving historical data, we dynamically adjust the information in the hidden state through an update gate $g_t$, effectively balancing the fusion of the student's historical interaction knowledge and current learning records. The calculation formula is as follows:

$$\begin{aligned} D_t &= \hat{X}_t \oplus \widetilde{h}_{t-1}, \\ g_t &= \sigma(W_q \cdot D_t + b_q), \end{aligned} \tag{12}$$

where $W_q \in \mathbb{R}^{(d_k+d_r+d_h) \times d_k}$ is the weight matrix of the update gate, $b_q \in \mathbb{R}^k$ is the bias vector, and $\sigma(\cdot)$ is the sigmoid activation function.

In order to ensure that the model can fully utilize the latest input information at the current time while considering the impact of historical information, we generate the candidate hidden state $\widetilde{h}_t$ at the current time and integrate the current input with historical information. This process enhances and updates the knowledge, allowing the model to dynamically balance the contributions of historical and new information. The calculation formula is as follows:

$$\begin{aligned} \widetilde{h}_t &= \tanh(W_h \cdot D_t + b_h), \\ h_t &= (1 - g_t) \cdot h + g_t \cdot \widetilde{h}_t, \end{aligned} \tag{13}$$

where $W_h \in \mathbb{R}^{(d_k+d_r+d_e+d_h) \times d}$ is the weight matrix for the candidate hidden state, $b_h \in \mathbb{R}^d$ is the bias vector, and $\cdot$ denotes element-wise multiplication.

Through the above gating mechanism, we achieve dynamic updates of the student's knowledge mastery, enabling the model to effectively integrate historical knowledge with new knowledge during the continuously evolving learning process.

### 4.5. Output layer

Through the KAG, we have obtained the learner's knowledge mastery at the current time step $t$, denoted as $h_t$. To better represent the student's performance in applying the learned knowledge at the next time step $t+1$, we integrate the exercise $e_{t+1}$ and the related knowledge component $kc_{t+1}$ at time step $t+1$ into the current cognitive state to make the output judgment. The formula is as follows:

$$\begin{aligned} y_t &= e_{t+1} \oplus kc_{t+1} \oplus h_t, \\ y_{t+1} &= \sigma(W_p \cdot y_t + b_p), \end{aligned} \tag{14}$$

where $W_p \in \mathbb{R}^{(d_k+d_e+d_h) \times d}$ is the learnable weight parameter, $b_p \in \mathbb{R}^d$ is the bias term, and $\sigma(\cdot)$ is the sigmoid nonlinear activation function.

### 4.6. Objective function

To optimize all parameters in the CEKT framework, we use cross-entropy as the objective function. This function evaluates the discrepancy between the framework's prediction $y_t$ and the system's acceptance rate $r$ of the student's submitted code. Specifically, the objective function is defined as follows:

$$\mathcal{L}(\beta) = -\sum_{t=1}^{T} \left(r_t \log(y_t) + (1 - r_t)\log(1 - y_t)\right) + \lambda_\beta \|\beta\|^2, \tag{15}$$

where $\lambda_\beta$ is the regularization hyperparameter used to control the complexity of the model parameters to avoid overfitting. To minimize the objective function $\mathcal{L}(\beta)$, we use the Adam optimizer. The specific experimental setup will be detailed in the subsequent experimental section.

## 5. Experiments

In this section, we first introduce the real-world dataset used in the experiments and describe in detail the data preprocessing, feature extraction, and specific details of the training process. Subsequently, we provide a detailed explanation of the baseline models used. Then, we compare the results of the baseline models with those of the

**Table 2**
Statistics of all datasets.

| Statistics \ Datasets | AIZU_Python | AIZU_Java | AIZU_C |
|---|---|---|---|
| Number of exercise records | 16,240 | 17,125 | 32,835 |
| Number of students | 369 | 367 | 855 |
| Number of programming problems | 1,493 | 1,516 | 1,709 |
| Average of exercise | 44.01 | 46.66 | 38.40 |
| Programming language types | Script | Object-oriented | Procedure |

proposed model in predicting students' future performance, showcasing the performance differences through various evaluation metrics. Additionally, to more comprehensively demonstrate the advantages and characteristics of the CEKT model, we delve into and answer the following research exercises. These exercises not only help us verify the effectiveness of the model but also reveal its potential and limitations in practical applications.

- RQ1: Does our proposed CEKT model have better performance in predicting students' future performance?
- RQ2: How does our proposed CEKT model impact the modeling of students' knowledge state?
- RQ3: How much prior knowledge should be given to LLMs to achieve optimal results?
- RQ4: How do the KCs generated by LLMs compare with those annotated by experts?
- RQ5: How adaptable are the KCs generated by LLM in different KT frameworks?
- RQ6: How do the different modules in CEKT influence the results of KT?
- RQ7: How does the CEKT model understand the code submitted by students?

### 5.1. Datasets

To validate the effectiveness of the CEKT framework, we selected three real-world datasets for testing. After cleaning the incomplete data, Table 2 presents detailed statistical information on the final filtered datasets. This dataset, obtained from AIZU.org, a popular platform among programming learners, includes a comprehensive collection of practice records from users of varying expertise, from beginners to advanced programmers. The problem sets are designed with an educational focus and cover a wide array of topics, ranging from simple exercises to more challenging, advanced algorithms [17]. In this study, we focus on the following representative programming languages: the high-level scripting language $Python$, the object-oriented programming language $Java$, and the procedural programming language $C$.

In online programming data, a single student may submit code for the same exercise multiple times. To more accurately assess students' programming skills, this study only retains the students' first submission data and focuses on analyzing data from students with more than 15 learning records. For the exercise lengths in the other datasets, we uniformly set the length to 50; sequences longer than this set length are split into multiple virtual students, while shorter sequences are padded with zeros.

### 5.2. Experimental setup

At the beginning of training, all model parameters were initialized randomly. The experimental environment included a 64-bit Ubuntu 20.04.5 LTS server, equipped with a 2.30 GHz Intel Xeon Gold 5218 CPU and a 32 GB Tesla-V100 GPU. The computation and model training were conducted under the PyTorch framework. This hardware configuration and software framework were chosen to ensure efficient computational performance and a stable training process, thereby fully validating the model's effectiveness and robustness.

To comprehensively evaluate the performance of the CEKT model, we conducted a comparative analysis with several existing baseline models. First, the dataset was divided into 80% for training and 20% for testing. During the hyperparameter tuning process, we further separated 20% of the training data as a validation set to optimize model performance and ensure a fair comparison. To achieve this, we precisely replicated these baseline models and meticulously tuned their parameters to ensure they operated at their optimal state, guaranteeing the fairness and scientific validity of the comparison. Additionally, we incorporated L2 regularization during training to prevent overfitting and employed an early stopping mechanism to avoid unnecessary computational costs or performance degradation due to prolonged training. Ultimately, we selected the average results from 5-fold cross-validation on the testing set as the final outcome. It is worth mentioning that our CEKT model uses GPT-4.0 as its LLM.

During the performance evaluation, we employed the Area Under the Curve (AUC) and Accuracy (ACC) as the primary performance metrics. The AUC assesses the model's classification ability across various thresholds, offering a nuanced view of its overall performance. ACC directly measures the correctness of the model's predictions, thereby providing a practical evaluation of its effectiveness in real-world applications.

- **DKT** [1] is a pioneering model in the field of KT, introducing deep learning techniques for the first time. It uses Recurrent Neural Networks (RNN) or Long Short-Term Memory (LSTM) units to infer students' understanding and mastery levels across different subjects.
- **DKVMN** [21] improves traditional memory-augmented neural networks by introducing a structured key–value memory system. The key part is used to locate relevant memory slots, while the value part is used to store and update information, enabling a more dynamic and detailed representation of students' knowledge state.
- **GKT** [11] introduces Graph Neural Networks (GNN) to KT for the first time by transforming the knowledge structure into a graphical form and redefining the KT task as a temporal node-level classification exercise within the GNN framework.
- **EKT** [47] evaluates the importance of each exercise relative to all KCs, considering the semantic nuances of exercises. We use CodeBERT [10] as the input embedding layer for this model.
- **Code-DKT** [13] is a code-based deep KT system that uses attention mechanisms to automatically extract and select domain-specific code features to enhance the DKT model. It uses code2vec [49] to embed students' submitted code.
- **PST** [14] captures learners' code characteristics when solving exercises by designing Code Information Graphs (CIG) and further develops Code Tracking Graphs (CTG) to measure changes between consecutive submissions.
- **DIMKT** [48] incorporates exercise difficulty levels into exercise representation and uses a hierarchical neural network to relate students' knowledge state to the changing difficulty of exercises over time.
- **QIKT** [8] skillfully tracks students' nuanced understanding changes by capturing cognitive representations of individual exercises and uses specialized knowledge acquisition and exercise-solving modules focused on these exercises.
- **TCKT** [10] introduces a causal self-attention mechanism based on front-door adjustment theory, enhancing interaction representation and mitigating prediction errors caused by inherent biases in the dataset.
- **THKT** [3] introduces target hierarchies into the KT task for the first time, identifying different target hierarchies for various students and then combining these hierarchies to output knowledge proficiency. In this paper, we only use the student behavior data related to the THKT model, which we have already obtained.

**Table 3**
Comparison of the AUC and ACC performance (%) of various KT methods on the AIZU_Python, AIZU_Java, and AIZU_C datasets. The best results are highlighted in **bold**, and the second-best results are underlined.

| Methods | Datasets | | | | | |
|---|---|---|---|---|---|---|
| | AIZU_Python | | AIZU_Java | | AIZU_C | |
| | AUC | ACC | AUC | ACC | AUC | ACC |
| DKT [1] | 71.95 | 69.80 | 71.87 | 66.62 | 71.07 | 69.81 |
| DKVMN [21] | 72.07 | 70.42 | 67.17 | 64.32 | 71.68 | 70.05 |
| GKT [11] | 73.32 | 70.05 | 72.78 | 67.21 | 72.02 | 70.94 |
| EKT [47] | 75.27 | 74.01 | 76.39 | 69.86 | 72.22 | 70.37 |
| Code-DKT [13] | 72.02 | 68.05 | 71.98 | 67.57 | 71.35 | 70.00 |
| PST [14] | 76.09 | 74.14 | 76.02 | 70.03 | 73.40 | 71.38 |
| DIMKT [48] | 74.01 | 72.04 | 76.06 | 68.91 | 72.88 | 71.95 |
| QIKT [8] | 72.36 | 71.15 | 72.54 | 67.47 | 73.69 | 72.00 |
| TCKT [10] | 72.79 | 72.01 | 73.31 | 68.57 | 73.82 | 71.11 |
| THKT [3] | 75.67 | 73.26 | 76.05 | 70.57 | 73.94 | 72.13 |
| CEKT | **77.65** | **75.35** | **77.86** | **70.64** | **78.35** | **76.72** |

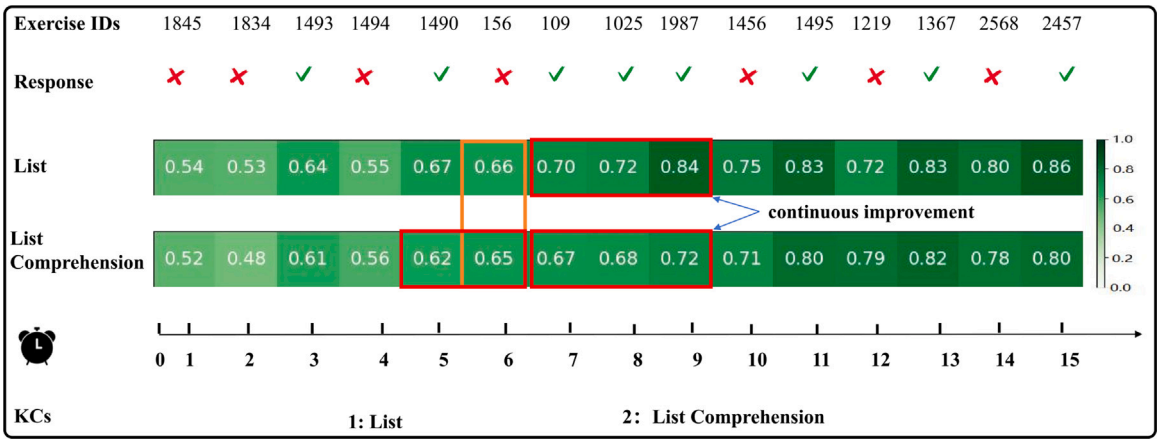



**Fig. 6.** The CEKT model tracks the evolution of a student's knowledge state over time as he/she solves programming exercises.

### 5.3. Performance prediction (RQ1)

The CEKT model constructs an accurate representation of a student's knowledge mastery by analyzing their historical exercise records. Our goal is not only to predict students' future performance but also to more accurately assess their knowledge mastery, thereby improving the precision of our predictions. To validate the effectiveness of the CEKT model in predicting future performance, we conducted a comparative analysis with existing baseline models. This comparative analysis allows us to comprehensively understand the performance of the CEKT model in evaluating students' knowledge mastery, thereby providing more effective personalized teaching support for students.

To ensure the accuracy of the evaluation results, we used two key metrics: Area Under the Curve (AUC) and Accuracy (ACC). Table 3 shows the statistically significant differences between our CEKT model and the best baseline models. Through analysis, we derived several important findings. First, the CEKT model significantly outperformed the baseline methods across all metrics, highlighting the potential of LLMs to enhance KT capabilities in the domain of code. Second, the model performed well across different programming subjects, demonstrating good adaptability. Particularly in the AIZU_C dataset, which has shorter exercise records, the model's performance improvement was most pronounced, proving its modeling advantage in handling shorter answer data that are more challenging for constructing student knowledge profiles. Finally, we found that models that encode exercise statements (such as EKT, PST, and THKT) generally perform better, indicating that the textual information of exercises is crucial for assessing students' knowledge mastery in programming practice. Since traditional models often directly use exercises as KCs [14], this result further demonstrates that fully modeling KCs can effectively enhance the model's capability, thereby improving its ability to represent students' knowledge state and predict their performance more accurately.

### 5.4. Knowledge state modeling (RQ2)

The accurate evaluation of students' knowledge states is crucial for KT. In programming exercises, a single exercise often encompasses multiple knowledge components (KCs). Capturing students' knowledge states in programming tasks through the CEKT model is essential for assessing their programming abilities. As illustrated in Fig. 6, the CEKT model depicts the state changes of students in two KCs that have a precursor relationship ("List" being a precursor to "List Comprehension"). The figure reveals several important phenomena.

Firstly, the model demonstrates that students' knowledge states change significantly as they progress through exercises. Secondly, for the same KC, the student's knowledge state continuously evolves with ongoing practice, indicating that our proposed CEKT model has a substantial impact on modeling students' knowledge states. For instance, while handling exercises 109, 1025, and 1987, from time steps 7 to 9, the student's knowledge state notably improved with consecutive correct code submissions. Thirdly, students' knowledge states are significantly influenced by precursor knowledge components (KCs). For example, the state change trend of the KC "List Comprehension" aligns with that of its precursor "List". However, a closer examination at time steps 5 and 6 reveals that the mastery level of "List Comprehension" is not solely dependent on the precursor KC. Despite a decrease in the mastery of "List" the mastery level of "List Comprehension" increases. This phenomenon occurs because "List Comprehension" relies not only on "List" but also on other precursor KCs, which showed improvement during this period. This indicates that the CEKT model offers good

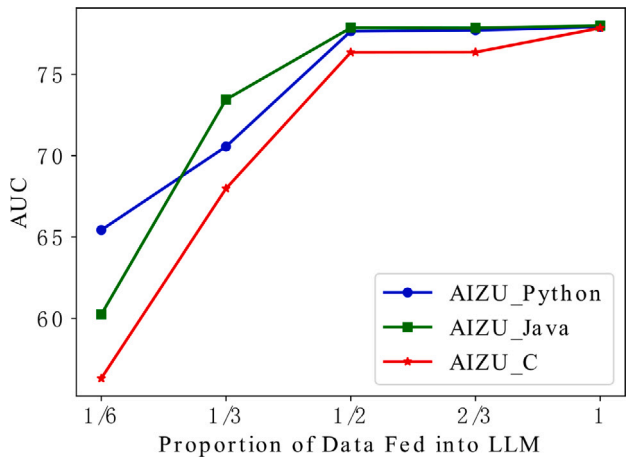



**Fig. 7.** By inputting different proportions of raw data into the LLMs to generate knowledge concepts, we can observe its impact on the model's AUC performance.

interpretability. This experiment validates the positive significance of our proposed use of LLMs to construct knowledge structures and their relationships for knowledge state modeling.

### 5.5. The impact of data proportion fed into LLMs on performance (RQ3)

Highly repetitive or similar knowledge input into LLMs may fail to yield new or useful information [50]. To assess the impact of varying proportions of the AIZU_Python, AIZU_Java, and AIZU_C datasets on the AUC performance of the CEKT model, we randomly selected different proportions $R$ ($R = \{1/6, 1/3, 1/2, 2/3, 1\}$, $r \in R$) of the complete training set for training. The results, as depicted in Fig. 7, reveal several key insights.

First, when $r = 1/2$, the AUC almost reaches its optimal effect, comparable to using the entire dataset for LLMs input. Therefore, to save resources, all experiments in this paper use $r = 1/2$ for domain knowledge generated by LLMs. Second, a closer examination reveals that when the dataset is insufficient, the model's performance is poor. This is mainly because the knowledge input into the LLMs is too sparse to accurately capture the required domain knowledge. Third, we also found that most programming exercises contain similar KCs, and it is not necessary to input all exercises into the model to achieve significant results. This indicates that the KCs generated by the LLMs can already effectively distinguish between different exercises. However, it also suggests that focusing solely on the KCs of the exercises, while ignoring other educational factors (such as exercise difficulty), may not allow for precise mastery assessment using LLMs-generated domain knowledge.

This experiment further illustrates that increasing the quantity of input data for LLMs does not necessarily yield the best domain knowledge. Two potential reasons are: first, overfitting, where the model may overlearn from a large volume of domain-specific information, capturing noise or anomalies in the dataset, which results in poorer performance on new, unseen data; and second, data quality issues, where the presence of irrelevant information means that more data does not equate to better data.

### 5.6. Comparing expert-annotated KCs and LLMs results (RQ4)

This experiment aims to compare the alignment between KCs manually annotated by experts and those automatically generated by LLM, with an additional comparison involving KC extensions generated using WordNet. We randomly selected 200 problems each from the AIZU_Python and AIZU_C question banks and invited five experienced programming language experts to annotate the KCs for these problems. For each problem, we selected the KC with the highest number of annotations as the consensus label. For KCs annotated by a few experts but supported by theoretical foundations, we retained and evaluated them based on domain knowledge to ensure the rationality and comprehensiveness of the annotations. Each problem was annotated with at least three KCs, denoted as $N$. The number of KCs automatically generated by the LLM is denoted as $M$, and the alignment is evaluated using the formula $KCC = \frac{N}{M} \times 100\%$.

To further expand the scope of KC generation, we used KeyBERT [51] to extract keywords from the problem descriptions and employed WordNet [52] to generate hypernyms and synonyms based on these keywords, thereby extending related KCs. Semantic similarity measures were used to comprehensively compare the alignment of KCs among expert annotations, LLMs-generated concepts, and WordNet-expanded concepts. The results are shown in Fig. 8.

Our study found that in the AIZU_Python dataset, the alignment rate between the LLM-generated KCs and expert annotations ranged from 61% to 100%, demonstrating good performance in extracting Python-related KCs. In the AIZU_C dataset, the alignment rate for most problems reached 81%–100%, indicating a high degree of consistency between the LLM-generated KCs and expert annotations. This result may be related to the structured and simplified nature of C language syntax, which enables the LLM to accurately generate relevant KCs. However, KCs generated through WordNet expansion showed lower alignment with expert annotations. Despite providing broader coverage, WordNet lacks the contextual understanding and semantic reasoning abilities that the LLM possesses. Overall, this result further validates the credibility and reasonableness of LLM-generated content to some extent, enhancing the interpretability of LLM applications in the field of education. Overall, this experiment highlights the significant potential of LLMs in KC generation, especially for structured and rule-based programming languages like C. However, for languages with higher semantic complexity and variability, such as Python, there is room for improvement. Incorporating WordNet for concept expansion revealed that semantic similarity measures could align relevant KCs to some extent. This finding suggests that future research could leverage the semantic understanding capabilities of LLMs along with the lexical expansion capabilities of WordNet to build a more comprehensive framework for joint KC generation, thereby providing stronger support for automated KC annotation in programming education.

### 5.7. The adaptability of LLM-generated KCs (RQ5)

We further embedded the LLM-generated KCs into several representative baseline models to validate their applicability and effectiveness in real-world educational settings. The experimental results are shown in Table 4. The results indicate that, firstly, the performance of these baseline models improved significantly, demonstrating that the KCs generated by LLMs have good generalization and applicability, thereby strongly validating their effectiveness. Secondly, upon closer analysis, it was found that for models like DKT and THKT, which directly reference KCs without further in-depth learning, the gains were significantly lower compared to models like GKT and EKT, which made deeper use of the KCs. This suggests that further exploration and utilization of LLM-generated KCs can significantly enhance their role in the models, and also further proves the importance of the CEKT model proposed in this study for KT.

### 5.8. Ablation experiment (RQ6)

To assess the impact of relationships between different types of KCs on model performance, we first started with a foundational KT model without including KCs generated by the large model (KAG w/o KCs). We then progressively added different types of relational edges (Key–Value, Value–Value, Key–Key). Experiments were conducted on three datasets (AIZU_Python, AIZU_Java, AIZU_C), and the model performance was evaluated using AUC and ACC metrics. The experimental results are shown in Table 5.

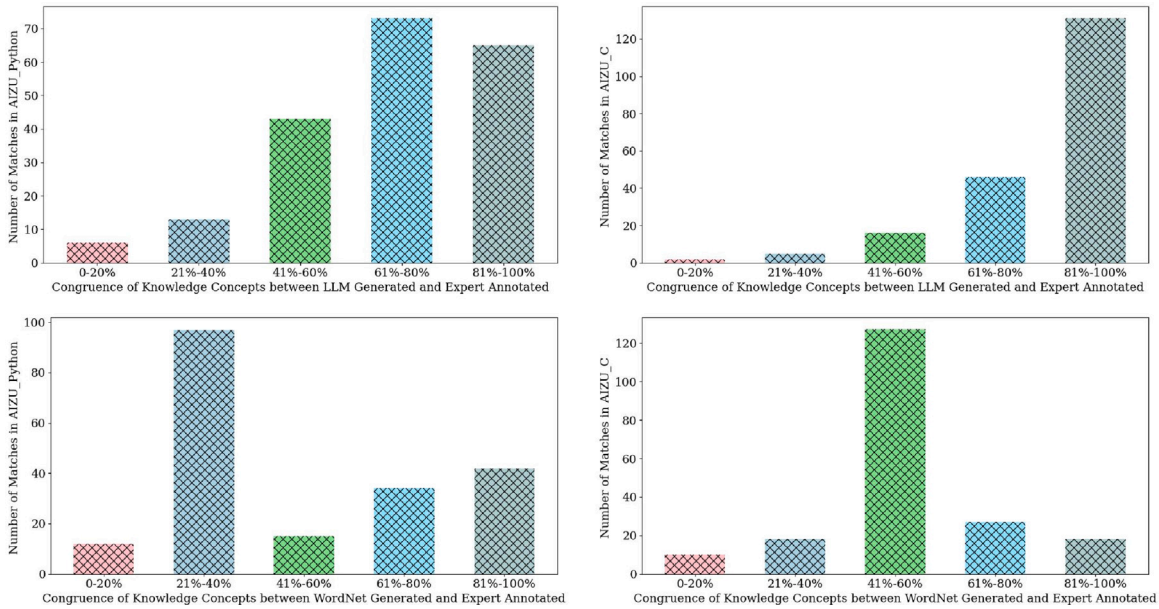



**Fig. 8.** Comparison of the alignment between Expert Annotated KCs and those generated by LLM and WordNet, respectively.

**Table 4**
Comparison of AUC Performance by Applying LLM-generated KCs from the AIZU_Python Dataset to Other Baselines.

| Baselines | + KCs | AIZU_Python | | |
|---|---|---|---|---|
| | | Original | After application | Gain |
| DKT | ✓ | 71.95 | 72.53 | +0.58 |
| GKT | ✓ | 73.32 | 74.57 | +1.25 |
| EKT | ✓ | 75.27 | 76.36 | +1.09 |
| THKT | ✓ | 75.67 | 76.24 | + 0.57 |

Through this experiment, we found that different types of relational edges have a significant impact on model performance improvement. The Key–Value relationship ($E_{kv}$) provided a slight performance boost in most metrics, although ACC showed a slight decline on the AIZU_C dataset. In contrast, the Value–Value relationship ($E_{vv}$) contributed the most to performance enhancement, primarily because a large number of fine-grained KCs are concentrated in the Value component. Additionally, incorporating the Key–Key relationship ($E_{kk}$) further elevated the model's performance to new heights. Ultimately, the CEKT model, which integrates all relational edges, outperformed other models across all datasets, fully demonstrating the powerful advantages and potential of generating KCs with LLMs and adopting multi-dimensional modeling. This study not only improves prediction accuracy but also provides new insights and methodologies for future educational big data analysis.

### *5.9. Visualizing student performance (RQ7)*

In this study, we utilized the performance of two students from the AIZU_Python dataset to explore the accuracy of diagnosing students' knowledge mastery through tracking knowledge state transitions. As shown in Fig. 9, this figure provides a visual explanation of how the CEKT model predicts students' future performance in programming KT.

Take student S1 as an example. This student correctly answered all three KCs in both exercise 1 and exercise 2, and the KCs in these two exercises happen to cover the KCs in exercise 3. Since the CEKT model takes into account the transition of knowledge state over time, subsequent exercises continually update the knowledge state. Therefore, at the stage of exercise 3, the KCs covered by the exercise are consistent with the KCs involved in previous exercises. Ultimately, the model's prediction perfectly matches the student's actual responses, demonstrating the high interpretability of the model.

However, student cognition is influenced by multiple factors, and the model does not always perform well in all cases. Particularly, since the answer records only provide the final answer to a exercise and do not consider the process scores of the exercise, if a exercise is answered incorrectly, the CEKT model cannot accurately filter out which KCs caused the incorrect answer. For instance, student S2 answered the same exercises as student S1, but due to an incorrect answer on the first exercise, it becomes exceptionally difficult to judge the student's current knowledge mastery level, ultimately leading to a mismatch between the predicted results and the actual answers.

Through this experiment, we can clearly see that a student's knowledge state is influenced by multiple factors, not just the relationships between KCs. Sufficient student answer records are crucial for KT tasks.

## 6. Conclusions and limitations

This paper proposes an innovative model that applies LLMs to programming KT. The model leverages the powerful code generation capabilities of LLMs to generate KCs in programming languages through prompts. It also models the relationships among these KCs in three dimensions: key–key, key–value, and value–value. The model uses a Graph Attention Network (GAT) to learn these relationships in order to assess students' programming abilities. Furthermore, by using prompts to extract the KCs required to solve specific exercises from the LLMs, and combining these KCs with their relationships to enhance student features, we can more accurately assess students' knowledge state and predict their performance on subsequent exercises through the KAG gating mechanism. Experimental results show that this model excels in knowledge state inference, accurately evaluating students' knowledge mastery. Additionally, by integrating explicitly generated KCs and their relationships from the LLMs, the model achieves better interpretability.

Although this study has made some progress, several limitations still require further exploration. First, the learning process of students involves complex educational psychology mechanisms, and merely tracking the dynamic changes in knowledge states is insufficient to comprehensively capture learning performance. Second, generating and evaluating KCs across different disciplinary contexts is crucial for verifying the applicability and generalizability of the proposed method. Therefore, future research could further explore the application of LLMs in assessing the difficulty of programming exercises and enhance

**Table 5**
The ablation study of different modules was conducted on three datasets. The "+ w" indicates adding modules to the KAG while also incorporating the GAT module, while '$KAGw/oKCs$' indicates removing KCs from the KAG.

| Model | AIZU_Python | | AIZU_Java | | AIZU_C | |
|---|---|---|---|---|---|---|
| | AUC | ACC | AUC | ACC | AUC | ACC |
| KAG w/o KCs | 71.93 | 69.77 | 71.88 | 66.52 | 71.05 | 69.91 |
| + $E_{kv}$ $w$ $GAT$ | 72.21 | 70.35 | 71.97 | 67.36 | 71.08 | 69.76 |
| + $E_{vv}$ $w$ $GAT$ | 76.93 | 75.21 | 76.58 | 69.84 | 78.21 | 75.39 |
| + $E_{kk}$ $w$ $GAT$ ⇒ CEKT | **77.65** | **75.35** | **77.86** | **70.64** | **78.35** | **76.72** |

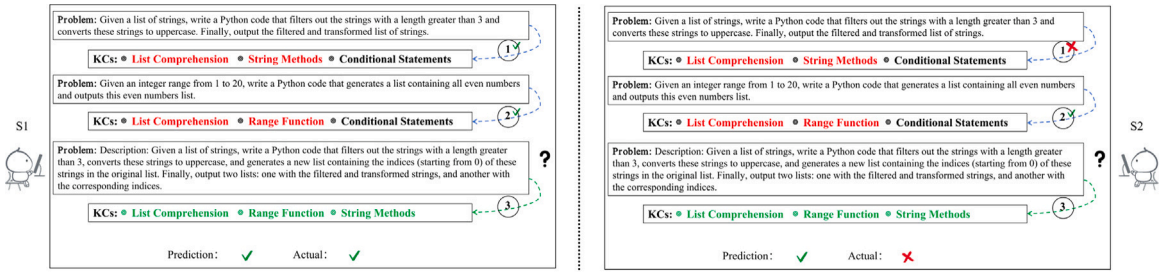



**Fig. 9.** Visual explanation of the CEKT model's prediction of future performance in programming knowledge tracking for two students under the same answer sequence segment.

the accuracy of KC extraction by integrating expert knowledge. Moreover, effectively identifying LLM-generated answers and preventing potential cheating behaviors remain key challenges that impact the practical application of this approach. Furthermore, improving the model's interpretability is essential for enhancing its transparency and credibility. Explainable AI methods can be employed to analyze the logical structure of KC generation and reveal the model's decision-making mechanisms through causal reasoning, thereby deepening the understanding of its inference process. In addition, challenges persist regarding the model's generalization and robustness. To address these issues, incorporating larger-scale and more diverse programming education datasets, along with data augmentation techniques, will be necessary to improve the model's adaptability and stability across different application scenarios.

## CRediT authorship contribution statement

**Xinjie Sun:** Writing – review & editing, Writing – original draft, Visualization, Software, Resources, Methodology. **Qi Liu:** Methodology, Funding acquisition. **Kai Zhang:** Funding acquisition, Formal analysis. **Shuanghong Shen:** Visualization, Methodology. **Lina Yang:** Software. **Hui Li:** Data curation.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgments

This research was partially supported by grants from the Guizhou Provincial Higher Education Undergraduate Teaching Content and Curriculum System Reform Project, China (GZJG2024331, GZJG2024323), Guizhou Provincial Science and Technology Projects, China (QKHJC[2024] Youth012), the Humanities and Social Sciences Research Project of Guizhou Provincial Universities, China (2024RW171), and the Guizhou Provincial Philosophy and Social Science Research Special Fund (Research on the Spirit of the National Education Conference), the China Postdoctoral Science Foundation under Grant 2024M760725, the National Natural Science Foundation of China (62337001, 62406303, 62261034, 62177031), the Guizhou Province First-Class Undergraduate Course, China (GZSy1kc202229), the Anhui Science and Technology Innovation Plan, China (202423k09020010), the Anhui Provincial Natural Science Foundation (2408085QF212, 2308085QF229), the Fundamental Research Funds for the Central Universities, China (WK2150110034).

## Appendix

**1: Prompt Template (Create a Relationship Graph Among Knowledge Concepts)**

You are an expert in programming languages and computer science education. According to the following knowledge elements definition:

*The "concept" refers to a specific programming topic, feature, or technique relevant to a particular language.*
*The "language" refers to the programming language context in which the concept is applied, such as Python, Java, or C.*
*The "concept details" refers to the explanation, examples, and significance of the specific concept within the chosen language.*

In the following task, there is a programming problem and a specified language. The problem may involve one concept or many concepts. You should analyze the problem and extract the "concept details" for each specific concept related to the language provided.

You can express the concept details in your own way, but remember you cannot change the "concept" word, as it is very important.

Remember, the concept may vary in complexity from basic to advanced. You should carefully analyze the problem and provide "concept details" based on your comprehension. Return NULL for fields and issues you are unfamiliar with.

Please give me your answer in the format as the following strictly:

**Format Example**
problem:
language:
concept:
concept details:

**2: Prompt Template (Constructing Code Domain Knowledge Concepts)**

You are an expert in programming languages and computer science education. Your task is to create a relationship graph among knowledge points based on the following relationship types:

1. *The relationship between keys and values.*
2. *The relationship between values and values.*
3. *The relationship between keys and keys.*

Each node in the graph represents a knowledge point, and the edges represent the relationships between them. You should carefully analyze the relationships and construct a graph that clearly illustrates these connections.

Remember, the nodes and edges should be labeled appropriately to convey the correct relationships. The complexity of the relationships may vary, so ensure that your graph is comprehensive and easy to understand.

**3: Prompt Template (Generating Knowledge Concepts and Q-matrix)**

You are an expert in educational assessment and knowledge representation. According to the following knowledge elements definition:

*The "Question" refers to a specific inquiry within a given domain.*
*The "Knowledge Concepts" refers to the key ideas, topics, or skills that are required to answer the question. Each question should involve no more than 5 knowledge points and no fewer than 1.*
*The "Q-matrix" is a binary matrix representing the relationship between questions and knowledge concepts, where each row corresponds to a question and each column corresponds to a knowledge concept.*

In the following task, there is a specified domain and a question. You should analyze the question and identify the relevant "Knowledge Concepts" associated with it. Then, construct the "Q-matrix" to represent these relationships.

Please provide your answer in the following format strictly:

**Format Example**

```
{
"Question": "What is the capital of France?"",
"Knowledge Concepts": [
"Geography",
"European Capitals",
"France"
]
}

{
"Q-matrix": [
 [0, 1, 0, ...], // Binary values for Question 1
 [1, 0, 1, ...], // Binary values for Question 2
...
 ]
}
```

## Data availability

Data will be made available on request.

## References


[1] Paul Denny, James Prather, Brett A Becker, James Finnie-Ansley, Arto Hellas, Juho Leinonen, Andrew Luxton-Reilly, Brent N Reeves, Eddie Antonio Santos, Sami Sarsa, Computing education in the era of generative AI, Commun. ACM 67 (2) (2024) 56–67.

[2] Shalini Pandey, George Karypis, A self-attentive model for knowledge tracing, 2019, arXiv preprint arXiv:1907.06837.

[3] Xinjie Sun, Kai Zhang, Shuanghong Shen, Fei Wang, Yuxiang Guo, Qi Liu, Target hierarchy-guided knowledge tracing: Fine-grained knowledge state modeling, Expert Syst. Appl. 251 (2024) 123898.

[4] Renny S.N. Lindberg, Teemu H. Laine, Lassi Haaranen, Gamifying programming education in K-12: A review of programming curricula in seven countries and programming games, Br. J. Educ. Technol. 50 (4) (2019) 1979–1995.

[5] Marcus Messer, Neil C.C. Brown, Michael Kölling, Miaojing Shi, Automated grading and feedback tools for programming education: A systematic review, ACM Trans. Comput. Educ. 24 (1) (2024) 1–43.

[6] M.M. Ashurova, M.U. Ashurov, The role and significance of the concepts of hard skill and soft skill in teaching it and programming languages, J. Pedagog. Inven. Pr. 18 (2023) 68–70.

[7] Sein Minn, Yi Yu, Michel C. Desmarais, Feida Zhu, Jill-Jenn Vie, Deep knowledge tracing and dynamic student classification for knowledge tracing, in: 2018 IEEE International Conference on Data Mining, ICDM, IEEE, 2018, pp. 1182–1187.

[8] Jiahao Chen, Zitao Liu, Shuyan Huang, Qiongqiong Liu, Weiqi Luo, Improving interpretability of deep sequential knowledge tracing models with question-centric cognitive representations, 2023, arXiv preprint arXiv:2302.06885.

[9] Aritra Ghosh, Neil Heffernan, Andrew S. Lan, Context-aware attentive knowledge tracing, in: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2020, pp. 2330–2339.

[10] Changqin Huang, Hangjie Wei, Qionghao Huang, Fan Jiang, Zhongmei Han, Xiaodi Huang, Learning consistent representations with temporal and causal enhancement for knowledge tracing, Expert Syst. Appl. (2024) 123128.

[11] Hiromi Nakagawa, Yusuke Iwasawa, Yutaka Matsuo, Graph-based knowledge tracing: modeling student proficiency using graph neural network, in: IEEE/WIC/ACM International Conference on Web Intelligence, 2019, pp. 156–163.

[12] Yang Yang, Jian Shen, Yanru Qu, Yunfei Liu, Kerong Wang, Yaoming Zhu, Weinan Zhang, Yong Yu, GIKT: a graph-based interaction model for knowledge tracing, in: Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2020, Ghent, Belgium, September 14–18, 2020, Proceedings, Part I, Springer, 2021, pp. 299–315.

[13] Yang Shi, Min Chi, Tiffany Barnes, Thomas Price, Code-DKT: A code-based knowledge tracing model for programming tasks, 2022, arXiv preprint arXiv:2206.03545.

[14] Ruixin Li, Yu Yin, Le Dai, Shuanghong Shen, Xin Lin, Yu Su, Enhong Chen, PST: measuring skill proficiency in programming exercise process via programming skill tracing, in: Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2022, pp. 2601–2606.

[15] Naiming Liu, Zichao Wang, Richard Baraniuk, Andrew Lan, Open-ended knowledge tracing for computer science education, in: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing, 2022.

[16] Renyu Zhu, Dongxiang Zhang, Chengcheng Han, Ming Gaol, Xuesong Lu, Weining Qian, Aoying Zhou, Programming knowledge tracing: A comprehensive dataset and a new model, in: 2022 IEEE International Conference on Data Mining Workshops, ICDMW, IEEE, 2022, pp. 298–307.

[17] Xinjie Sun, Qi Liu, Kai Zhang, Shuanghong Shen, Yan Zhuang, Yuxiang Guo, LGS-KT: Integrating logical and grammatical skills for effective programming knowledge tracing, Neural Netw. (2025) 107164.

[18] Shuanghong Shen, Qi Liu, Zhenya Huang, Yonghe Zheng, Minghao Yin, Minjuan Wang, Enhong Chen, A survey of knowledge tracing: Models, variants, and applications, IEEE Trans. Learn. Technol. (2024).

[19] Ghodai Abdelrahman, Qing Wang, Bernardo Nunes, Knowledge tracing: A survey, ACM Comput. Surv. 55 (11) (2023) 1–37.

[20] Xinjie Sun, Kai Zhang, Qi Liu, Shuanghong Shen, Fei Wang, Yuxiang Guo, Enhong Chen, DASKT: A dynamic affect simulation method for knowledge tracing, IEEE Trans. Knowl. Data Eng. (2025).

[21] Jiani Zhang, Xingjian Shi, Irwin King, Dit-Yan Yeung, Dynamic key-value memory networks for knowledge tracing, in: Proceedings of the 26th International Conference on World Wide Web, 2017, pp. 765–774.

[22] Ghodai Abdelrahman, Qing Wang, Knowledge tracing with sequential key-value memory networks, in: Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval, 2019, pp. 175–184.

[23] Xia Sun, Xu Zhao, Bo Li, Yuan Ma, Richard Sutcliffe, Jun Feng, Dynamic key-value memory networks with rich features for knowledge tracing, IEEE Trans. Cybern. 52 (8) (2021) 8239–8245.

[24] Lap Pong Cheung, Haiqin Yang, Heterogeneous features integration in deep knowledge tracing, in: Neural Information Processing: 24th International Conference, ICONIP 2017, Guangzhou, China, November 14-18, 2017, Proceedings, Part II 24, Springer, 2017, pp. 653–662.

[25] Shashank Sonkar, Andrew E. Waters, Andrew S. Lan, Phillip J. Grimaldi, Richard G. Baraniuk, Qdkt: Question-centric deep knowledge tracing, 2020, arXiv preprint arXiv:2005.12442.
[26] Shuanghong Shen, Enhong Chen, Qi Liu, Zhenya Huang, Wei Huang, Yu Yin, Yu Su, Shijin Wang, Monitoring student progress for learning process-consistent knowledge tracing, IEEE Trans. Knowl. Data Eng. (2022).
[27] Shiwei Tong, Qi Liu, Wei Huang, Zhenya Hunag, Enhong Chen, Chuanren Liu, Haiping Ma, Shijin Wang, Structure-based knowledge tracing: An influence propagation view, in: 2020 IEEE International Conference on Data Mining, ICDM, IEEE, 2020, pp. 541–550.
[28] Shuyuan Xu, Zelong Li, Kai Mei, Yongfeng Zhang, CoRE: LLM as interpreter for natural language programming, pseudo-code programming, and flow programming of AI agents, 2024, arXiv preprint arXiv:2405.06907.
[29] Yuzhe Cai, Shaoguang Mao, Wenshan Wu, Zehua Wang, Yaobo Liang, Tao Ge, Chenfei Wu, Wang You, Ting Song, Yan Xia, et al., Low-code llm: Visual programming over llms, 2 (2023) arXiv preprint arXiv:2304.08103.
[30] Breanna Jury, Angela Lorusso, Juho Leinonen, Paul Denny, Andrew Luxton-Reilly, Evaluating llm-generated worked examples in an introductory programming course, in: Proceedings of the 26th Australasian Computing Education Conference, 2024, pp. 77–86.
[31] Shraddha Barke, Emmanuel Anaya Gonzalez, Saketh Ram Kasibatla, Taylor Berg-Kirkpatrick, Nadia Polikarpova, HYSYNTH: Context-free LLM approximation for guiding program synthesis, 2024, arXiv preprint arXiv:2405.15880.
[32] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, Brad Myers, Using an llm to help with code understanding, in: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, 2024, pp. 1–13.
[33] Anisha Agarwal, Aaron Chan, Shubham Chandel, Jinu Jang, Shaun Miller, Roshanak Zilouchian Moghaddam, Yevhen Mohylevskyy, Neel Sundaresan, Michele Tufano, Copilot evaluation harness: Evaluating LLM-guided software programming, 2024, arXiv preprint arXiv:2402.14261.
[34] Juo-Tung Chen, Chien-Ming Huang, Forgetful large language models: Lessons learned from using LLMS in robot programming, in: Proceedings of the AAAI Symposium Series, Vol. 2, No. 1, 2023, pp. 508–513.
[35] Majeed Kazemitabaar, Runlong Ye, Xiaoning Wang, Austin Zachary Henley, Paul Denny, Michelle Craig, Tovi Grossman, Codeaid: Evaluating a classroom deployment of an llm-based programming assistant that balances student and educator needs, in: Proceedings of the CHI Conference on Human Factors in Computing Systems, 2024, pp. 1–20.
[36] Aysa Xuemo Fan, Ranran Haoran Zhang, Luc Paquette, Rui Zhang, Exploring the potential of large language models in generating code-tracing questions for introductory programming courses, 2023, arXiv preprint arXiv:2310.15317.
[37] Yang Yu, Yingbo Zhou, Yaokang Zhu, Yutong Ye, Liangyu Chen, Mingsong Chen, ECKT: Enhancing code knowledge tracing via large language models, in: Proceedings of the Annual Meeting of the Cognitive Science Society, Vol. 46, 2024.
[38] Lingyue Fu, Hao Guan, Kounianhua Du, Jianghao Lin, Wei Xia, Weinan Zhang, Ruiming Tang, Yasheng Wang, Yong Yu, SINKT: A structure-aware inductive knowledge tracing model with large language model, 2024, arXiv preprint arXiv:2407.01245.
[39] Annapurna Vadaparty, Daniel Zingaro, David H. Smith IV, Mounika Padala, Christine Alvarado, Jamie Gorson Benario, Leo Porter, CS1-LLM: Integrating LLMs into CS1 instruction, in: Proceedings of the 2024 on Innovation and Technology in Computer Science Education V. 1, 2024, pp. 297–303.
[40] Majeed Kazemitabaar, Xinying Hou, Austin Henley, Barbara Jane Ericson, David Weintrop, Tovi Grossman, How novices use LLM-based code generators to solve CS1 coding tasks in a self-paced learning environment, in: Proceedings of the 23rd Koli Calling International Conference on Computing Education Research, 2023, pp. 1–12.
[41] Teemu Lehtinen, Charles Koutcheme, Arto Hellas, Let's ask AI about their programs: Exploring ChatGPT's answers to program comprehension questions, in: Proceedings of the 46th International Conference on Software Engineering: Software Engineering Education and Training, 2024, pp. 221–232.
[42] Xinjie Sun, Kai Zhang, Qi Liu, Meikai Bao, Yanjiang Chen, Harnessing domain insights: A prompt knowledge tuning method for aspect-based sentiment analysis, Knowl.-Based Syst. 298 (2024) 111975.
[43] Kirsti M. Ala-Mutka, A survey of automated assessment approaches for programming assignments, Comput. Sci. Educ. 15 (2) (2005) 83–102.
[44] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, Denny Zhou, Self-consistency improves chain of thought reasoning in language models, 2022, arXiv preprint arXiv:2203.11171.
[45] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, Yusuke Iwasawa, Large language models are zero-shot reasoners, Adv. Neural Inf. Process. Syst. 35 (2022) 22199–22213.
[46] John R. Anderson, Acquisition of cognitive skill, Psychol Rev 89 (4) (1982) 369.
[47] Qi Liu, Zhenya Huang, Yu Yin, Enhong Chen, Hui Xiong, Yu Su, Guoping Hu, Ekt: Exercise-aware knowledge tracing for student performance prediction, IEEE Trans. Knowl. Data Eng. 33 (1) (2019) 100–115.
[48] Shuanghong Shen, Zhenya Huang, Qi Liu, Yu Su, Shijin Wang, Enhong Chen, Assessing student's dynamic knowledge state by exploring the question difficulty effect, in: Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2022, pp. 427–437.
[49] Uri Alon, Meital Zilberstein, Omer Levy, Eran Yahav, Code2vec: Learning distributed representations of code, Proc. ACM Program. Lang. 3 (POPL) (2019) 1–29.
[50] Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Hamza Alobeidli, Alessandro Cappelli, Baptiste Pannier, Ebtesam Almazrouei, Julien Launay, The RefinedWeb dataset for Falcon LLM: Outperforming curated corpora with web data only, Adv. Neural Inf. Process. Syst. 36 (2023) 79155–79172.
[51] Jill Sammet, Ralf Krestel, Domain-specific keyword extraction using BERT, in: Proceedings of the 4th Conference on Language, Data and Knowledge, 2023, pp. 659–665.
[52] George A. Miller, WordNet: a lexical database for English, Commun. ACM 38 (11) (1995) 39–41.